Maryam Irshad

Env Protection Site

6th July 2026

OVERVIEW

The idea is to make .env files secured from malicious threats, As of now most people share their .env files on different sites like **slack, whatsapp, gmail,** etc where they are vulnerable to attacks. By replacing manual secret distribution with authenticated, centralized access control, the system improves collaboration, reduces the risk of accidental secret exposure, and provides capabilities that are difficult to achieve with traditional .env file workflows.

GOALS

1. Securely store encrypted environment variables.
2. Eliminate manual sharing of .env files.
3. Authenticate users before granting access.
4. Allow administrators to revoke developer access instantly.
5. Provide temporary access sessions.
6. Read one time only policy

Technical Stack:

Frontend

- React
- Next.js
- Tailwind CSS

Backend

- Node.js

Database

- Supabase

Authentication

- JWT
- OAuth (Google/GitHub)
- Multi-factor authentication (future enhancement)

Encryption

- AES-256 encryption
- TLS for data transmission

Operational Feasibility

The system is designed for software developers working individually or collaboratively while ensuring that sensitive environment variables remain confidential throughout the development lifecycle.

Users can:

- Create and manage projects.
- Upload `.env` files or individual environment variables.
- Encrypt secrets locally within the browser before transmission.
- Invite collaborators to projects.
- Assign role-based permissions (e.g., Owner, Editor, Viewer).
- Securely share project access without exposing plaintext secrets.
- Retrieve and decrypt project secrets locally after successful authentication.

Administrators/Project Owners can:

- Add or remove collaborators.
- Revoke project access instantly.
- Manage project permissions.
- Configure session expiration policies.

The proposed system minimizes manual secret sharing while providing a familiar workflow similar to modern collaboration platforms such as GitHub. Since encryption and decryption occur within the user's browser, no additional software installation is required, making the system practical for both individuals and development teams.

Security Feasibility

The proposed system adopts a **Zero-Knowledge Architecture**, ensuring that sensitive environment variables are encrypted within the user's browser before being transmitted to the server. Consequently, the server stores only encrypted data and cannot access or decrypt user secrets, even if the database is compromised.

Existing Approach

Developer A

Shares .env file through Discord / Slack / Email / WhatsApp

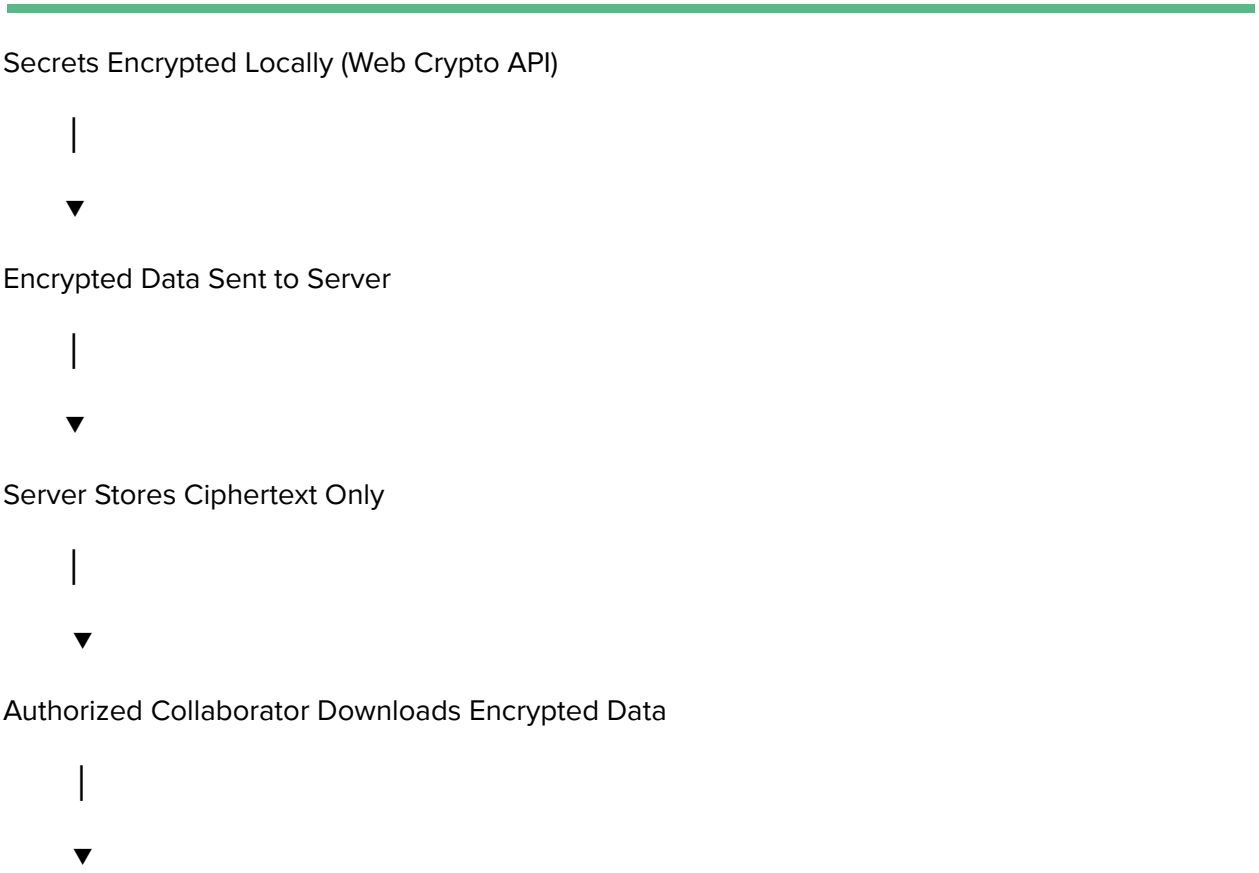
Developer B

Proposed Zero-Knowledge Approach

Developer

Login & Identity Verification

Browser Generates/Unlocks Encryption Keys



In this architecture, the server is responsible for authentication, authorization, project management, and storing encrypted data. It never possesses users' private keys or the plaintext project secrets.

Security Features

- **Zero-Knowledge Architecture** (server cannot read stored secrets).
- **Client-side encryption** using the **Web Crypto API**.
- **AES-256-GCM** for encrypting project secrets.
- **Public-key cryptography** for securely sharing project encryption keys among collaborators.
- **HTTPS/TLS** for secure communication.
- **Role-based access control (RBAC)**.
- **Encrypted storage** of project secrets in the database.
- **Immediate access revocation** for collaborators.
- Configurable session expiration.